

# 自动驾驶小车H题

## 1. 开篇说明

请先阅读四路电机驱动板资料中的《电机介绍以及用法》，了解清楚自己现使用的电机参数、接线方式、供电电压。以免造成烧坏主板或者电机的后果。

使用提示：由于MPU6050的偏航角容易漂移，或者是使用了不同的电机，此工程在烧录后可能不能完美在赛道上运行，可以根据下面代码解析中提到的一些关键点，去修改变量，或者将MPU6050自行换成其他的模块。

## 2. 实验准备

国赛底盘V2四驱版本、4\*L520电机、12V锂电池、八路巡线模块、MPU6050、MSPM0机器人扩展板（选配）、MSPM0G3507核心板（亚博）。H题中要求的声光提醒分别集成在MSPM0机器人扩展板（蜂鸣器 PB24）、亚博版MSPM0G3507核心板（LED PB2）上，若未使用MSPM0机器人扩展板的用户需要自备一个蜂鸣器。以及赛题选择功能按键K1（PA2），位于MSPM0机器人扩展板上

**4个电机接口对应小车的关系如下：**

M1 -> 左上电机(小车的左前轮)

M2 -> 左下电机(小车的左后轮)

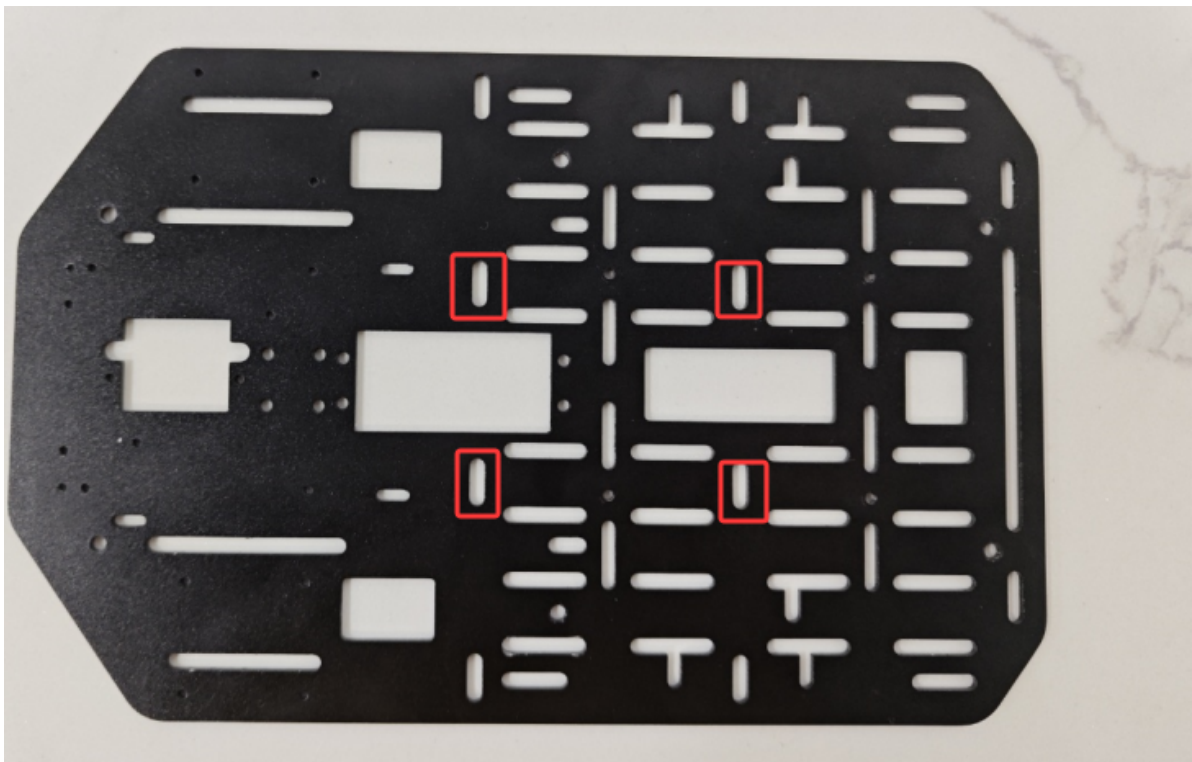
M3 -> 右上电机(小车的右前轮)

M4 -> 右下电机(小车的右后轮)

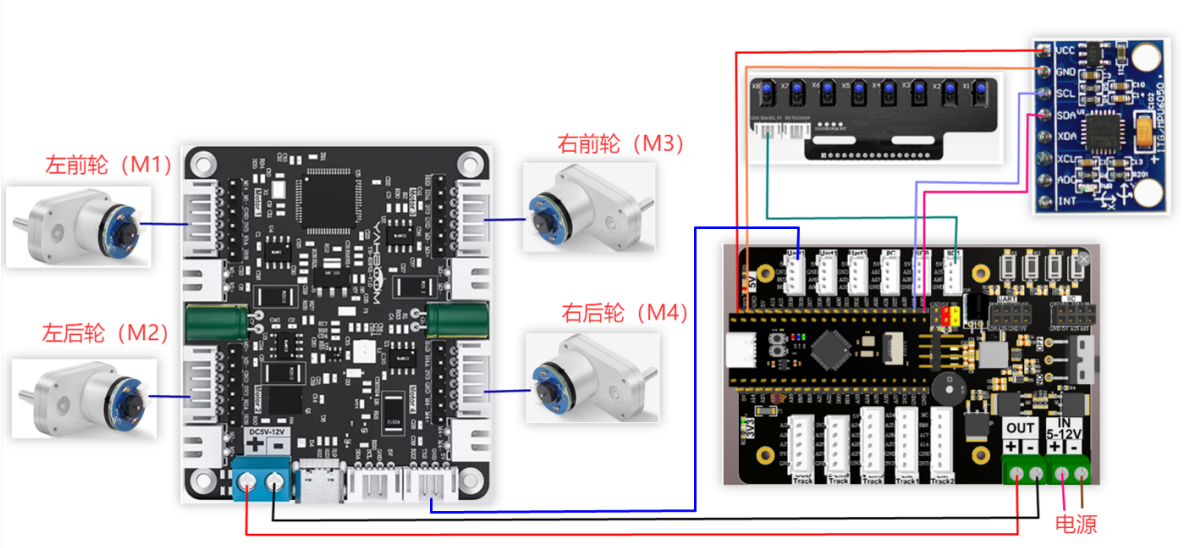
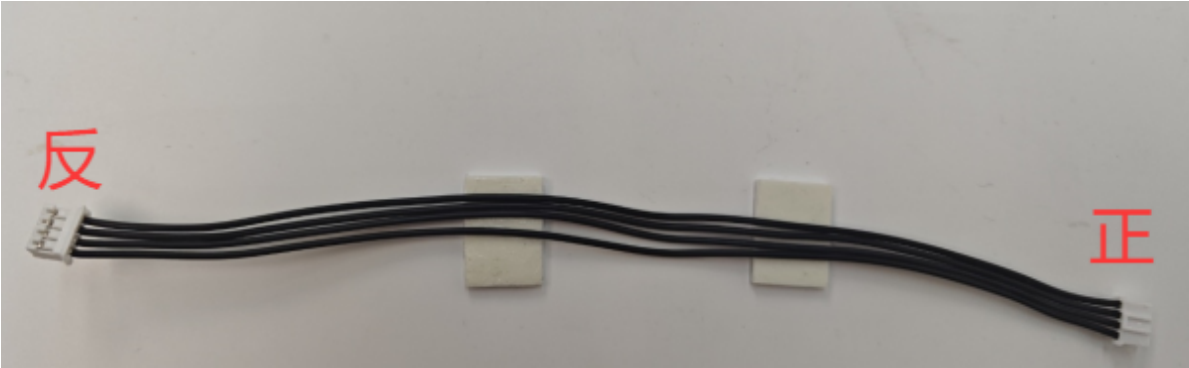
**硬件接线：**

### 使用mspm0扩展板接线

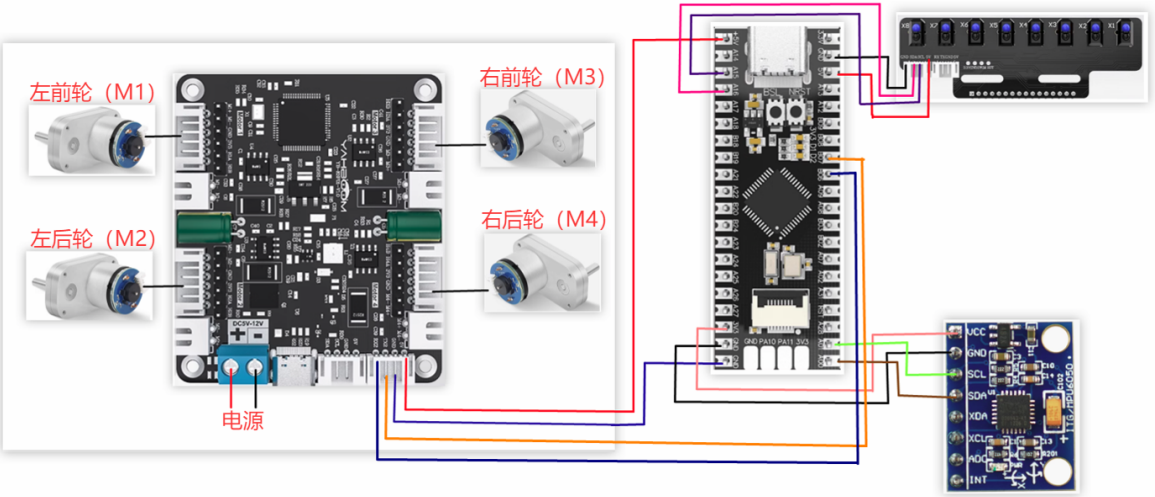
在安装接线过程中，如果发现接线长度不够，可以把MSPM0机器人扩展板往前移一点安装，如下图位置。



注意：八路巡线模块使用的线材、MSPM0机器人扩展板与四路电机驱动模块连接所用的线材为：PH2.0-4pin排线 双头全黑 反向(200mm)，反向排线座子方向如下图所示



使用msp核心板接线



接线引脚：

| 四路电机驱动板 | MSPM0G3507 |
|---------|------------|
| RX2     | PB6        |
| TX2     | PB7        |
| GND     | GND        |
| 5V      | 5V         |

| 电机 | 四路电机驱动板(Motor) |
|----|----------------|
| M2 | M-             |
| V  | 3V3            |
| A  | H1B            |
| B  | H1A            |
| G  | GND            |
| M1 | M+             |

| 八路巡线模块 | MSPM0G3507 |
|--------|------------|
| 5V     | 5V         |
| SCL    | PA15       |
| SDA    | PA16       |
| GND    | GND        |

| MPU6050 | MSPM0G3507 |
|---------|------------|
| VCC     | 5V         |
| GND     | GND        |
| SCL     | PA1        |
| SDA     | PA0        |

### 3.关键代码解析

根据自己所购买的电机型号，在empty.c开头处先修改为自己的电机型号，否则让小车运行时容易失控。

- empty.c

```
#define MOTOR_TYPE 5 //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L
型520电机
//1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT
DC reduction motor 5:L type 520 motor
```

MOTOR\_TYPE：用于设置使用的电机类型，根据自己现使用的电机对照着注释修改对应的数字。

由于MPU6050运行过程中，容易出现角度的偏移，以及不同电机的编码器数据会不一样。那么在运行赛题题目的时候，就需要根据自身情况修改 question.c 文件下，判断角度与里程数的变量。这里以题目3为例，解释一些可能需要修改的地方

- question.c

```
void Question_Task_3(void)
{
    if(q3_first_flag == 0)
    {
        //记录第一次发车的角度    Record the angle of the first departure
        first_angle = calibratedYaw;
        //开始计算里程数    Start calculating mileage
        encoder_odometry_flag = 1;
        //声光提示    Sound and light prompts
        bee_time = 300;
        //开始问题3的小状态机    Start the small state machine of problem 3
        State_Machine.Q3_State = Q3_STATE_1;

        q3_first_flag = 1;
    }
    if(State_Machine.Q3_State == Q3_STATE_1)
    {
        //设定斜切目标角度    Set the beveled target angle
        object_angle = navigation_0_360_limit(first_angle + 32);

        Motion_Car_Control(300,0,0);
        Angle_error = get_minor_arc(object_angle,calibratedYaw);
        PID_Value = Dir_PID(Angle_error);
        Motion_Yaw_Calc(PID_Value);

        if(LineCheck() == BLACK && odometry_sum >= 1000)
        {
            bee_time = 300;
            //停止计算里程数并清零    Stop calculating mileage and clear
            encoder_odometry_flag = 0;
            odometry_sum = 0;
            //进入问题3的下一个状态    Enter the next state of question 3
            State_Machine.Q3_State = Q3_STATE_2;
        }
    }
    else if(State_Machine.Q3_State == Q3_STATE_2)    //问题3的状态2(左转灰度循迹)
    State 2 of question 3 (left turn grayscale tracking)
    {
        Linewalking();

        //检测到白线，同时角度在设定的范围内时，进入下一个状态    when the white line is
        detected and the angle is within the set range, it enters the next state
        if((calibratedYaw < 220 && calibratedYaw > 150) && LineCheck() ==
        WHITE)
        {
            bee_time = 300;
            State_Machine.Q3_State = Q3_STATE_3;
        }
    }
}
```

```

else if(State_Machine.Q3_State == Q3_STATE_3)
{
    static int Q3_state3_first_flag = 0;
    if(Q3_state3_first_flag == 0)
    {
        //设定斜切目标角度 Set the beveled target angle
        object_angle =navigation_0_360_limit(first_angle + 180) - 11;
        Q3_state3_first_flag = 1;
    }
    Motion_Car_Control(300,0,0);
    //开始计算里程数 Start calculating mileage
    encoder_odometry_flag = 1;
    Angle_error = get_minor_arc(object_angle,calibratedYaw);
    PID_Value = Dir_PID(Angle_error);
    Motion_Yaw_Calc(PID_Value);

    //检测到黑线，并且里程计数值大于等于1200时进入下一个状态 when the black line is
    detected and the mileage count value is greater than or equal to 1200, enters the
    next state
    if(odometry_sum >= 1200 && LineCheck() == BLACK)
    {
        //停止计算里程数并清零 Stop calculating mileage and clear
        encoder_odometry_flag = 0;
        odometry_sum = 0;
        beeper_time = 300;
        Q3_state3_first_flag = 0;
        State_Machine.Q3_State = Q3_STATE_4;
    }
}
else if(State_Machine.Q3_State == Q3_STATE_4)
{
    Linewalking();
    //检测到白线，同时角度在设定的范围内时，进入下一个状态 when the white line is
    detected and the angle is within the set range, it enters the next state
    if((calibratedYaw > 325 || calibratedYaw < 90) && LineCheck() ==
WHITE)
    {
        beeper_time = 300;
        State_Machine.Q3_State = Q3_STATE_5;
    }
}
else
{
    //退出当前题目 Exit the current question
    State_Machine.Q3_State = STOP_STATE;
    State_Machine.Main_State = STOP_STATE;
    //问题3的运行标志位清零 The running flag bit of question 3 is cleared
    q3_first_flag = 0;
}
}

```

在Q3\_STATE\_1中: `object_angle = navigation_0_360_limit(first_angle + 32);`是设置小车运行时要旋转的角度的，这里是出发角度+32°。如果自己在运行的时候，32°无法到达对应的点位，就根据自己小车偏移的情况修改32这个数字。

if(LineCheck() == BLACK && odometry\_sum >= 1000) 这里是利用八路巡线传感器检测有没有到达黑线，而且里程计数有没有大于等于1000.如果赛道条件不好，八路巡线传感器误触了，导致提前进入下一个状态，就可以将 odometry\_sum 调大。

在Q3\_STATE\_2中: if((calibratedYaw < 220 && calibratedYaw > 150) && LineCheck() == WHITE)，这里利用了MPU6050传来的角度，判断小车是否处于150°-220°的范围区间，而且已经到达白底区域。但是有时候小车到达这个点的时候，角度可能会超出这个范围，导致小车不能进入下一个状态，这时候就可以选择修改这里的角度值，具体修改多少需要自己实测。

- empty.c

```
#define MOTOR_TYPE 5    //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
                        //1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC reduction motor 5:L type 520 motor

int main(void)
{
    USART_Init();//打印串口初始化、四路电机通信串口初始化    Print serial port initialization, four-channel motor communication serial port initialization

    //设置电机类型    Set motor type
    Set_Motor(MOTOR_TYPE);
    send_upload_data(false,true,false);delay_ms(10);

    /*MPU6050初始化 MPU6050 Initialization*/
    MPU6050_Init();
    //DMP初始化 DMP Initialization
    while( mpu_dmp_init() )
    {
        printf("dmp error\r\n");
        delay_ms(200);
    }

    //蜂鸣器初始化 Buzzer initialization
    PWM_Buzzer_Init();
    //问题状态机初始化 Problem state machine initialization
    State_Machine_init();

    //清除定时器中断标志 Clear timer interrupt flag
    NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
    //使能定时器中断 Enable Timer Interrupt
    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
    //定时器开始计时 Timer start
    DL_TimerA_startCounter(TIMER_0_INST);

    //初始化成功后，LED灯快闪两下，蜂鸣器快响两下    After the initialization is successful, the LED light flashes twice, and the buzzer rings twice
    int i = 0;
    for(i=0;i<4;i++)
    {
        LED2_Toggle();
        Buzzer_Toggle();
        delay_ms(100);
    }
}
```

```

}

while(1)
{
    Scheduler_Run();//开始运行任务调度器    Start running the task scheduler

    //题目选择判断    Question selection judgment
    if(g_LinePortal_flag)
    {
        switch(State_Machine.Main_State)
        {
            case STOP_STATE:
                Contrl_Pwm(0,0,0,0);
                break;

            case QUESTION_1://赛题1 Question 1
                Question_Task_1();
                break;

            case QUESTION_2://赛题2 Question 2
                Question_Task_2();
                break;

            case QUESTION_3://赛题3 Question 3
                Question_Task_3();
                break;

            case QUESTION_4://赛题4 Question 4
                Question_Task_4();
                break;

            default:
                break;
        }
    }
}
}
}
}

```

先将所有外设都进行初始化，成功后LED灯快闪两下，蜂鸣器快响两下。然后运行任务调度器，按顺序获取所需的数据。while循环中持续判断有没有通过按键来选择赛题。

- MOTOR\_TYPE: 修改成自己在使用的电机类型
- app\_motor.c

```

void Set_Motor(int MOTOR_TYPE)
{
    if(MOTOR_TYPE == 1)
    {
        send_motor_type(1);//配置电机类型    Configure motor type
        delay_ms(100);
    }
}

```



```

        send_pulse_phase(30); //配置减速比 查电机手册得出    Configure the reduction
ratio. Check the motor manual to find out
        delay_ms(100);
        send_pulse_line(11); //配置磁环线 查电机手册得出    Configure the magnetic ring
wire. Check the motor manual to get the result.
        delay_ms(100);
        send_wheel_diameter(67.00); //配置轮子直径,测量得出    Configure the
wheel diameter and measure it
        delay_ms(100);
        send_motor_deadzone(1900); //配置电机死区,实验得出    Configure the motor dead
zone, and the experiment shows
        delay_ms(100);
    }

    else if(MOTOR_TYPE == 2)
    {
        send_motor_type(2);
        delay_ms(100);
        send_pulse_phase(20);
        delay_ms(100);
        send_pulse_line(13);
        delay_ms(100);
        send_wheel_diameter(48.00);
        delay_ms(100);
        send_motor_deadzone(1600);
        delay_ms(100);
    }

    else if(MOTOR_TYPE == 3)
    {
        send_motor_type(3);
        delay_ms(100);
        send_pulse_phase(45);
        delay_ms(100);
        send_pulse_line(13);
        delay_ms(100);
        send_wheel_diameter(68.00);
        delay_ms(100);
        send_motor_deadzone(1600);
        delay_ms(100);
    }

    else if(MOTOR_TYPE == 4)
    {
        send_motor_type(4);
        delay_ms(100);
        send_pulse_phase(48);
        delay_ms(100);
        send_motor_deadzone(1000);
        delay_ms(100);
    }

    else if(MOTOR_TYPE == 5)
    {
        send_motor_type(1);
        delay_ms(100);
        send_pulse_phase(40);
        delay_ms(100);
    }

```



```
    send_pulse_line(11);  
    delay_ms(100);  
    send_wheel_diameter(67.00);  
    delay_ms(100);  
    send_motor_deadzone(1900);  
    delay_ms(100);  
}  
}
```

这个函数用于存储本店在售电机的参数，通过修改empty.c开头处的MOTOR\_TYPE参数，就可以实现一键配置。正常情况下使用本店的电机不要修改此处的代码。如果使用的是自己的电机，就可以任选一个电机，将参数都改成自己所使用的电机的参数。如果不理解这些参数的含义，可以查看文档《4路电机驱动板控制指令》来了解每一项指令的具体含义。

## 4.实验现象

---

小车上电之后。使用到的外设会进行初始化操作。需要稍等一会，初始化成功后，LED会快速闪动两次，蜂鸣器快速蜂鸣两下，然后就可以长按住扩展板上的K1按键，进行赛题的选择，听到滴的一声为第一题，滴两声为第二题，以此类推。选择好题目之后，就再短按下K1按键开始运行任务，蜂鸣器会长鸣一声，然后开始运行任务。